

Introduction to Machine Learning

CI52

Dr. A N Ramya Shree
Dept. of CSE(AI&ML)

What is ML?

- Machine learning evolved as a subfield of artificial intelligence (AI) involving self-learning algorithms that derive knowledge from data to make predictions.
- Machine learning offers a more efficient alternative for capturing the knowledge in data to gradually improve the performance of predictive models and make data-driven decisions.

- **"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."**
- **Three key aspects of machine learning:**
 - Task (T): The specific job or activity that the algorithm is expected to perform.**
 - Experience (E): The data or interactions that the algorithm uses to improve its performance.**
 - Performance (P): A measure or metric used to assess how well the algorithm is completing the task.**

Case study:

Spam email classification

- Task (T): Classifying emails as spam or not spam.
- Experience (E): The set of labeled emails (spam or not spam).
- Performance (P): The accuracy or other metrics of classification on future emails.

- **Machine learning is about using data and experience to enhance the system's ability to perform tasks better over time**

Three types of machine learning

Supervised
learning

Unsupervised
learning

Reinforcement
learning



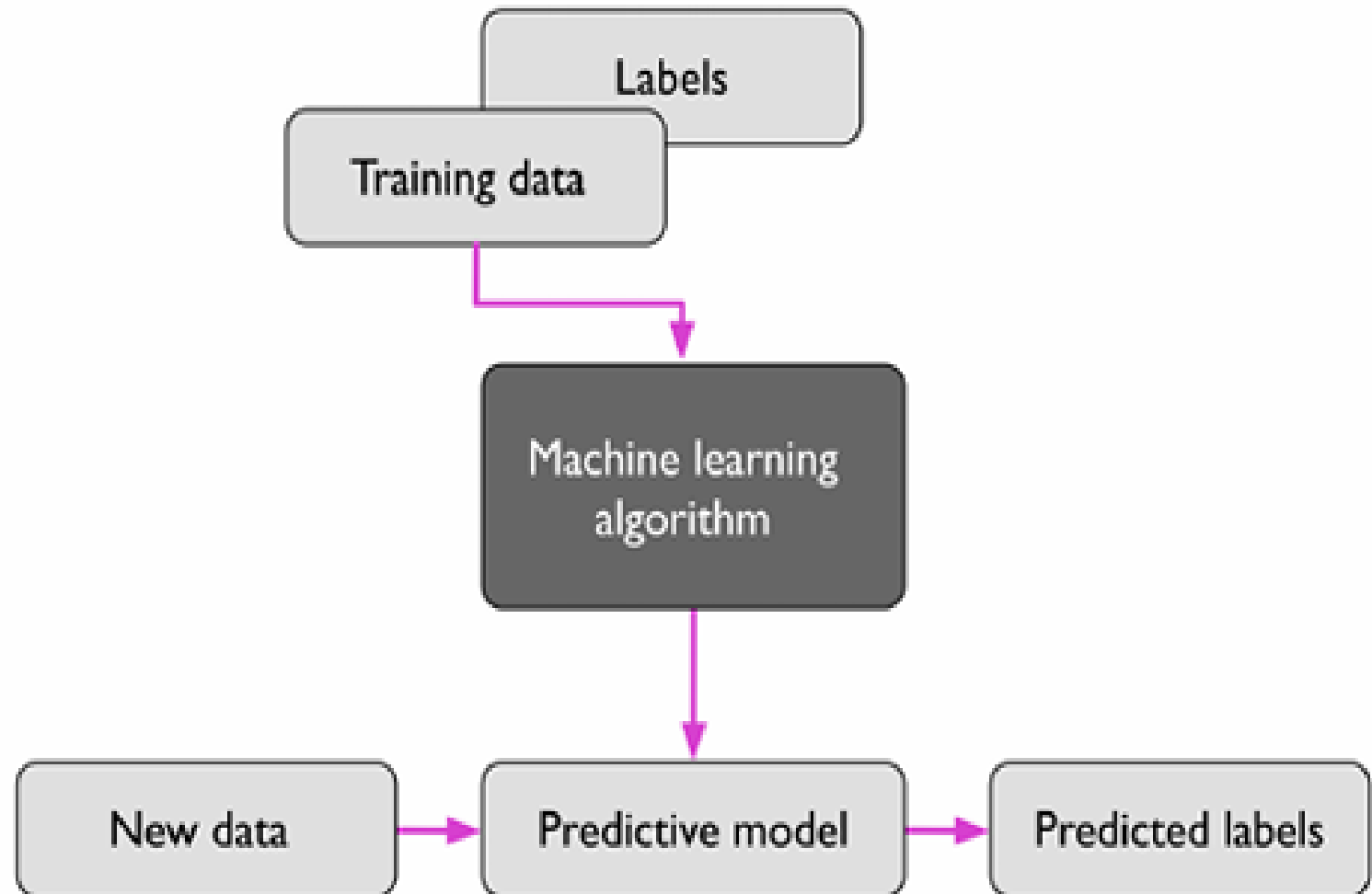
Three types of machine learning

Supervised learning	> Labeled data > Direct feedback > Predict outcome/future
Unsupervised learning	> No labels/targets > No feedback > Find hidden structure in data
Reinforcement learning	> Decision process > Reward system > Learn series of actions

Supervised Learning

- The main goal in supervised learning is to learn a model from labeled training data that allows us to make predictions about unseen or future data.
- Here, the term “supervised” refers to a set of training examples (data inputs) where the desired output signals (labels) are already known.
- Supervised learning is then the process of modeling the relationship between the data inputs and the labels.
- We can also think of supervised learning as **“label learning.”**

Supervised Learning Process



Regression

- **Regression** is a type of supervised machine learning technique used for predicting a continuous outcome or value based on input data. It establishes a relationship between the dependent variable (target) and one or more independent variables (features) to predict future values.
- The goal of regression is to fit a model to the data that can describe the relationship and minimize the error between the predicted and actual values.



- A second type of supervised learning is the prediction of continuous outcomes, which is also called regression analysis.
- In regression analysis, we are given a number of predictor (explanatory) variables and a continuous response variable (outcome), and we try to find a relationship between those variables that allows us to predict an outcome.
- In the field of machine learning, the predictor variables are commonly called “features,” and the response variables are usually referred to as “target variables.”
- For example, let’s assume that we are interested in predicting the math SAT scores of students. (The SAT is a standardized test frequently used for college admissions in the United States.)
 - If there is a relationship between the time spent studying for the test and the final scores, we could use it as training data to learn a model that uses the study time to predict the test scores of future students who are planning to take this test.

linear regression

- Given a feature variable, x , and a target variable, y , we fit a straight line to this data that minimizes the distance most commonly the average squared distance between the data points and the fitted line.
- We can now use the intercept and slope learned from this data to predict the target variable of new data.

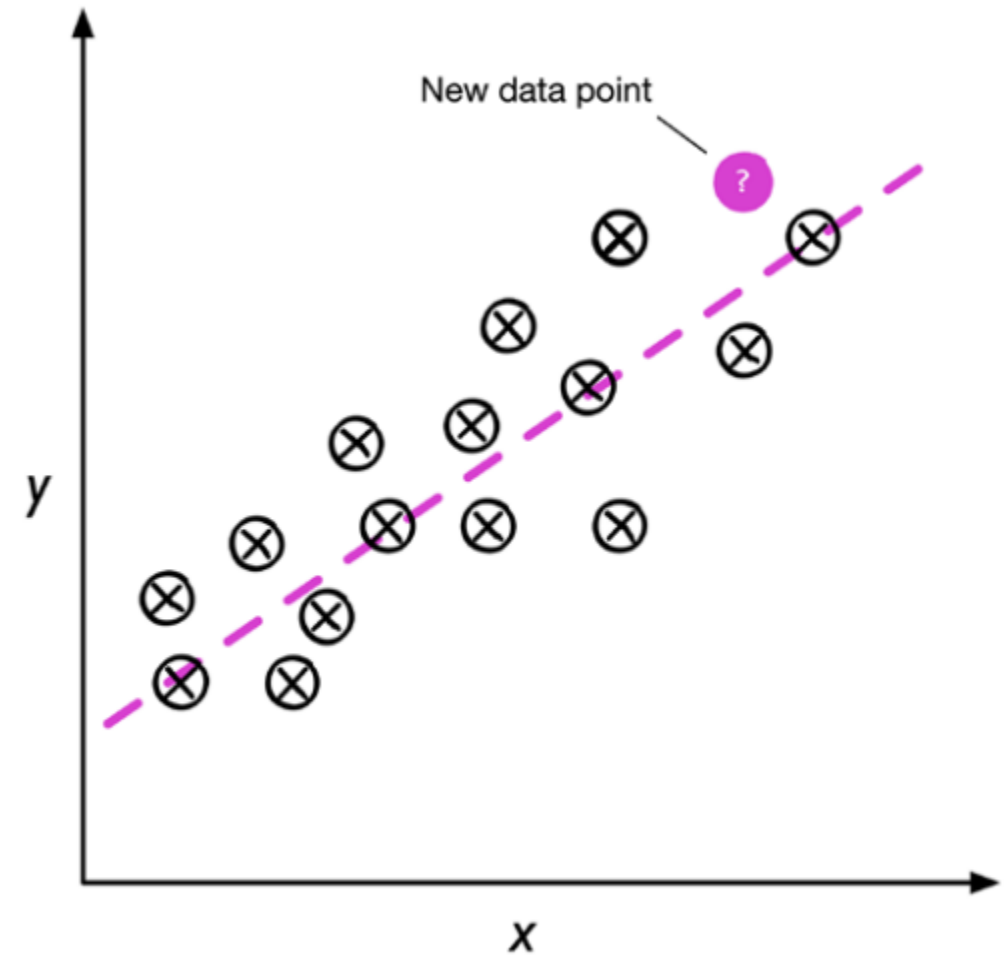


Figure 1.4: A linear regression example

- Linear Regression is a fundamental machine learning algorithm used for predictive analysis. It establishes a relationship between a dependent variable (target) and one or more independent variables (predictors) by fitting a straight line to the data.
- This line is represented by the equation:
 - $y=mx+c$ (for simple linear regression)
 - $y=\beta_0+\beta_1x_1+\beta_2x_2+...+\beta_nx_n$ (for multiple linear regression).
 - Dependent Variable (y): The output or value we are trying to predict.
 - Independent Variables (x): The inputs or features used to make predictions.
 - Parameters (m,c or $\beta_0,\beta_1,...,\beta_n$): These represent the coefficients and intercept of the line.
 - Goal: Minimize the sum of squared residuals $\sum(y_i - \hat{y}_i)^2$ where $\hat{y}_i$ is the predicted value.

Use cases



PREDICTING HOUSE PRICES
BASED ON VARIOUS FACTORS
LIKE SIZE, LOCATION ETC.



FORECASTING STOCK MARKET
TRENDS.



ESTIMATING AN INDIVIDUAL'S
INCOME BASED ON AGE,
EDUCATION, AND EXPERIENCE.

Classification

- **Classification** is a type of supervised machine learning task where the goal is to assign a label or category to a given input based on the input data.
- In classification, the output variable (also called the dependent variable) is categorical, meaning it falls into one of several predefined classes.

Use cases



Email filtering (spam vs. non-spam).



Medical diagnosis
(classifying diseases based
on symptoms).



Image recognition
(classifying objects in
images).

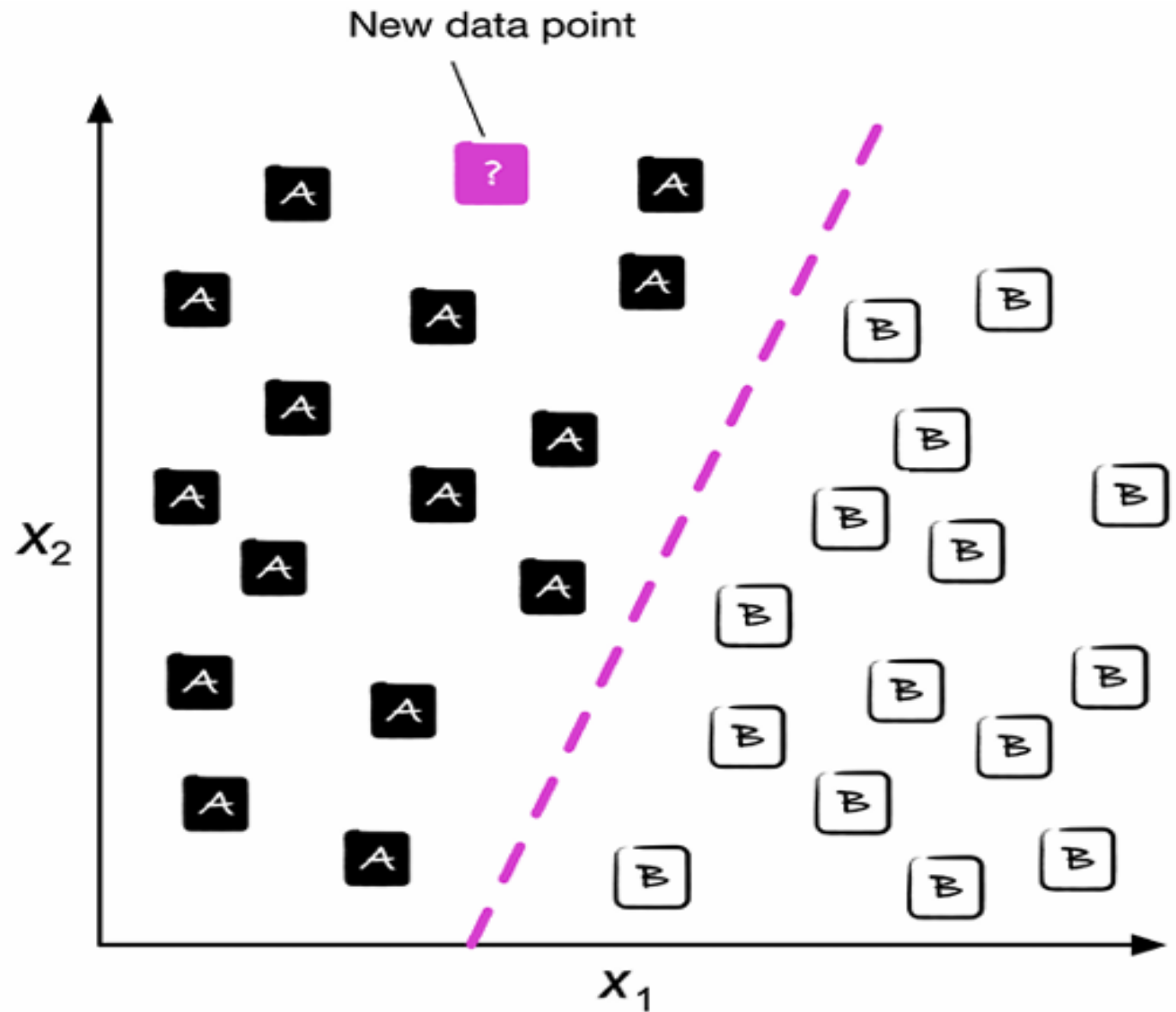


Sentiment analysis
(classifying text as positive,
negative, or neutral).

Classification for predicting class labels

- Classification is a subcategory of supervised learning where the goal is to predict the categorical class labels of new instances or data points based on past observations.
- Those class labels are discrete, unordered values that can be understood as the group memberships of the data points.
- email spam detection represents a typical example of a binary classification task, where the machine learning algorithm learns a set of rules to distinguish between two possible classes: **spam and non-spam emails**.

Classifying a
new data
point



Concept of a binary classification task

- Given 30 training examples, 15 training examples are labeled as class A and 15 training examples are labeled as class B.
- In this scenario, our dataset is two-dimensional, which means that each example has two values associated with it: x_1 and x_2 .
- We can use a supervised machine learning algorithm to learn a rule the decision boundary represented as a dashed line that can separate those two classes and classify new data into each of those two categories given its x_1 and x_2 values.
- The set of class labels does not have to be of a binary nature.
- The predictive model learned by a supervised learning algorithm can assign any class label that was presented in the training dataset to a new, unlabeled data point or instance.

- A typical example of a multiclass classification task is handwritten character recognition.
- We can collect a training dataset that consists of multiple handwritten examples of each letter in the alphabet.
- The letters (“A,” “B,” “C,” and so on) will represent the different unordered categories or class labels that we want to predict.
- Now, if a user provides a new handwritten character via an input device, our predictive model will be able to predict the correct letter in the alphabet with certain accuracy.
- However, our machine learning system will be unable to correctly recognize any of the digits between 0 and 9, for example, if they were not part of the training dataset.

Reinforcement Learning

- In reinforcement learning, the goal is to develop a system (agent) that improves its performance based on interactions with the environment.
- Since the information about the current state of the environment typically also includes a so-called reward signal, we can think of reinforcement learning as a field related to supervised learning.
- However, in reinforcement learning, this feedback is not the correct ground truth label or value, but a measure of how well the action was measured by a reward function.
- Through its interaction with the environment, an agent can then use reinforcement learning to learn a series of actions that maximizes this reward via an exploratory trial-and-error approach or deliberative planning.

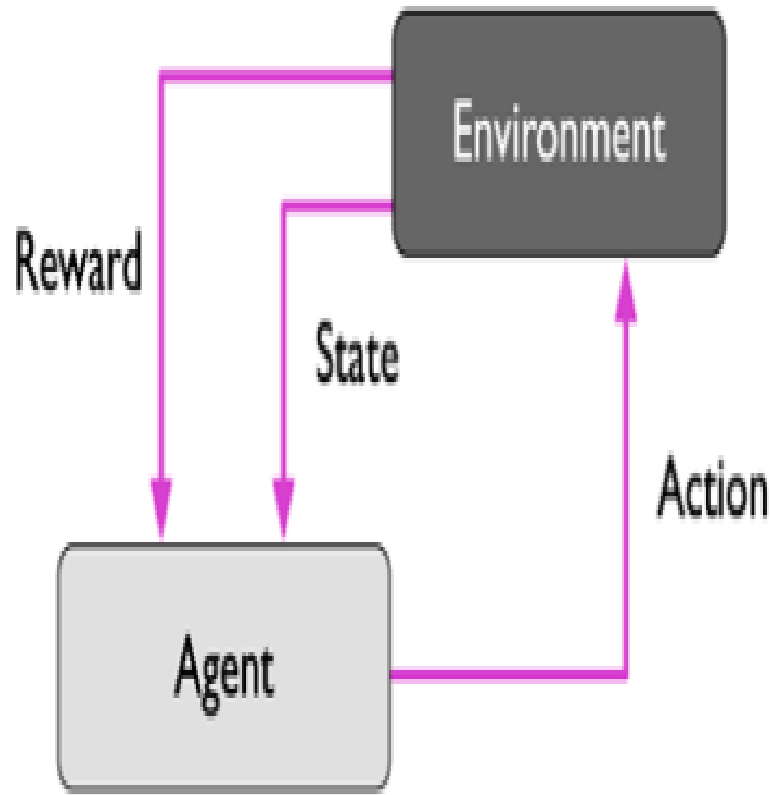


Figure 1.5: Reinforcement learning process

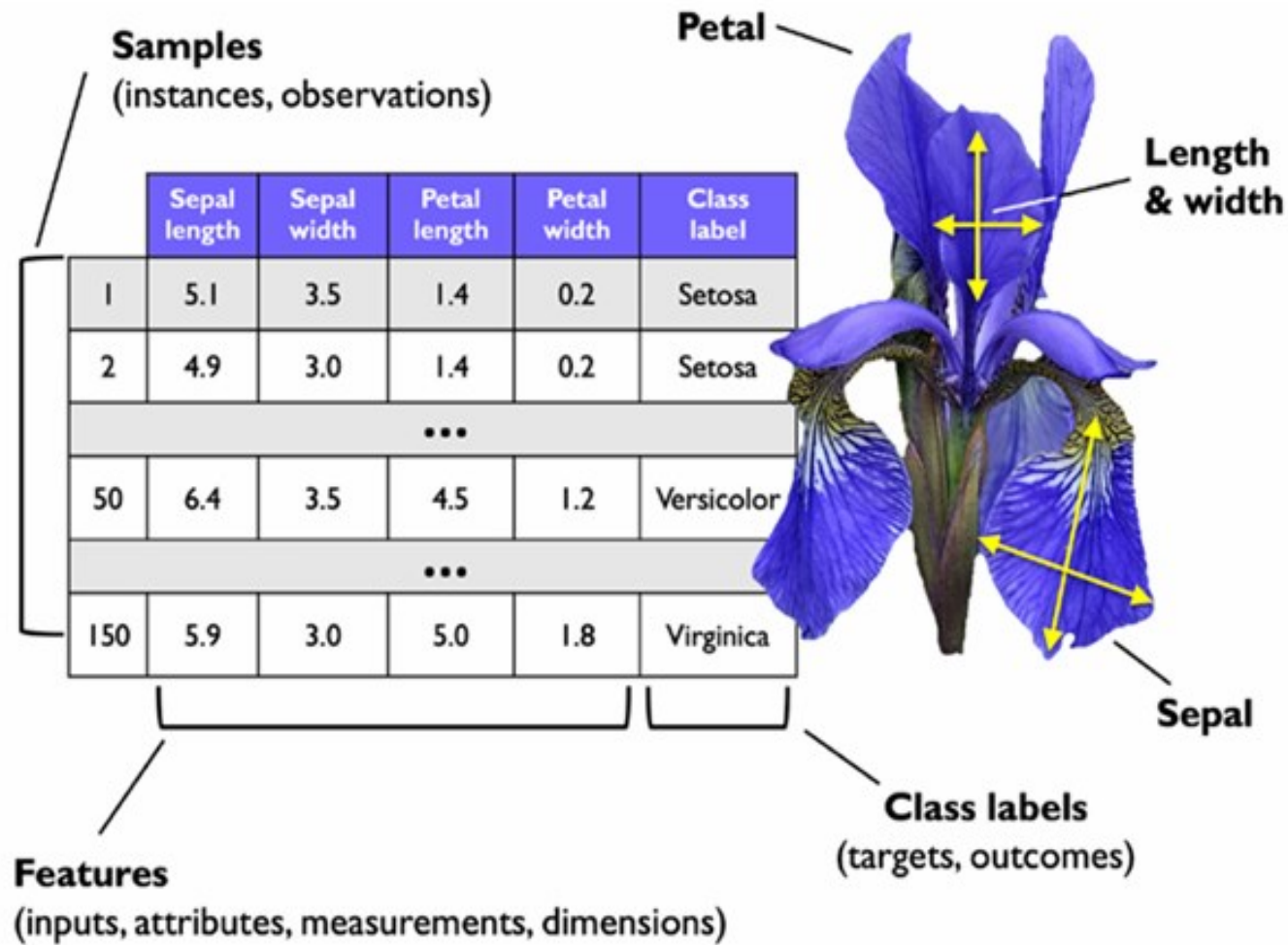
- The agent in reinforcement learning tries to maximize the reward through a series of interactions with the environment.
- Each state can be associated with a positive or negative reward, and a reward can be defined as accomplishing an overall goal, such as winning or losing a game of chess.
- For instance, in chess, the outcome of each move can be thought of as a different state of the environment.
- Reinforcement learning is concerned with learning to choose a series of actions that maximizes the total reward, which could be earned either immediately after taking an action or via delayed feedback



Use case

- A popular example of reinforcement learning is a chess program.
- Here, the agent decides upon a series of moves depending on the state of the board (the environment), and the reward can be defined as win or lose at the end of the game.
- Let's think of visiting certain configurations on the chessboard as being associated with states that will more likely lead to winning—for instance, removing an opponent's chess piece from the board or threatening the queen.
- Other positions, however, are associated with states that will more likely result in losing the game, such as losing a chess piece to the opponent in the following turn.
- Now, in the game of chess, the reward (either positive for winning or negative for losing the game) will not be given until the end of the game. In addition, the final reward will also depend on how the opponent plays.
- For example, the opponent may sacrifice the queen but eventually win the game.

Iris Flower



The Iris dataset, consisting of 150 examples and four features, can then be written as a 150×4 matrix, formally denoted as $X \in \mathbb{R}^{150 \times 4}$:

$$\begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & x_4^{(1)} \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & x_4^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(150)} & x_2^{(150)} & x_3^{(150)} & x_4^{(150)} \end{bmatrix}$$



Machine Learning- Terminologies

- **Training example:** A row in a table representing the dataset and synonymous with an observation, record, instance, or sample (in most contexts, sample refers to a collection of training examples).
- **Training:** Model fitting, for parametric models like parameter estimation.
- **Feature(x):** A column in a data table or data (design) matrix. Synonymous with predictor, variable, input, attribute, or covariate.
- **Target(y):** Synonymous with outcome, output, response variable, dependent variable, (class) label, and ground truth.
- **Loss function:** Often used synonymously with a cost function. Sometimes the loss function is also called an error function.
- **Cost** is a measurement that computes the loss (average or summed) over the entire dataset.

building
machine
learning
systems

Predictive Modeling Workflow

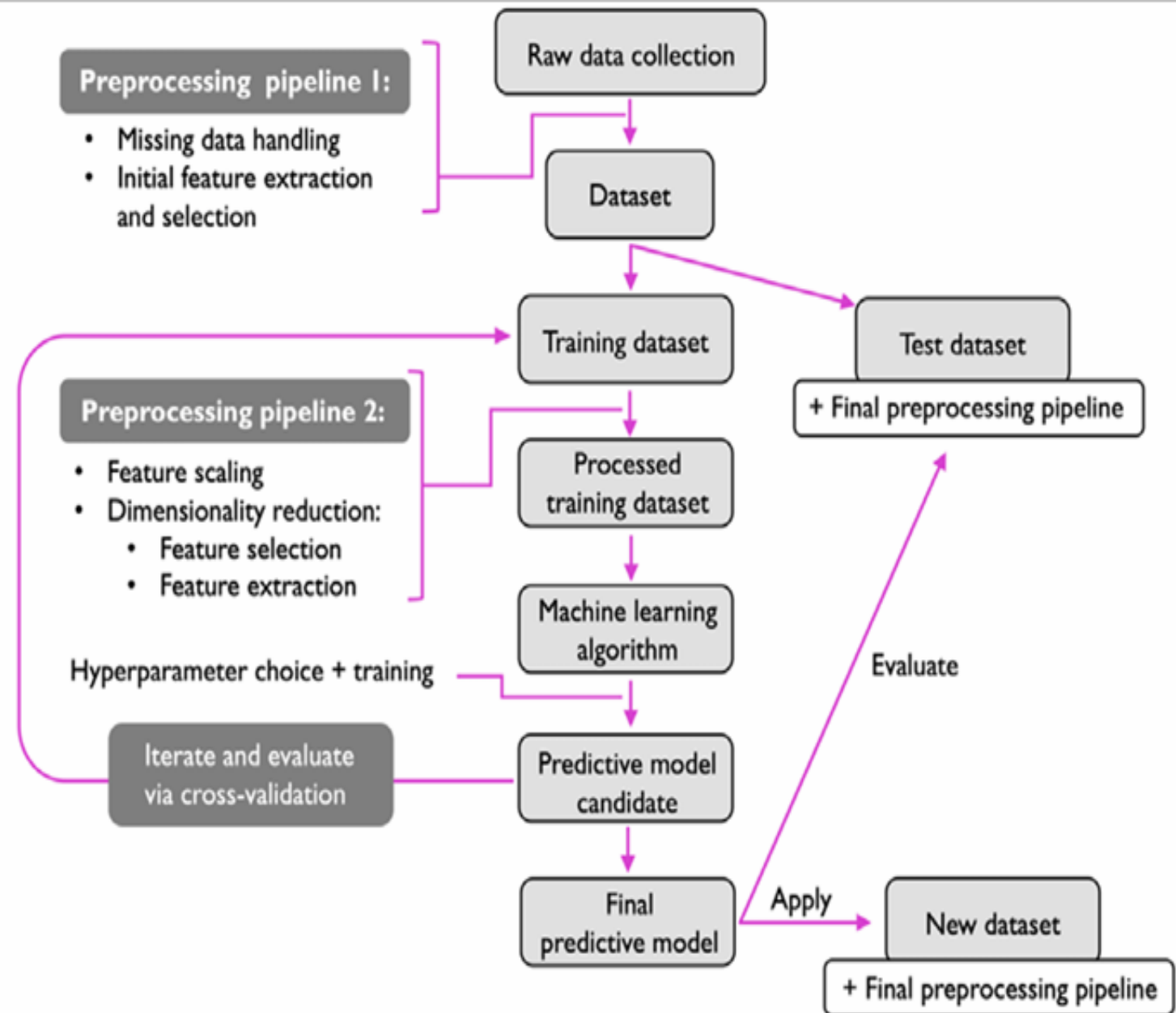


Figure 1.9: Predictive modeling workflow

1. Preprocessing – getting data into shape

- Raw data rarely comes in the form and shape that is necessary for the optimal performance of a learning algorithm. Thus, the preprocessing of the data is one of the most crucial steps in any machine learning application.
- If we take the Iris flower dataset from the previous section as an example, we can think of the raw data as a series of flower images from which we want to extract meaningful features. Useful features could be centered around the color of the flowers or the height, length, and width of the flowers.
- Many machine learning algorithms also require that the selected features are on the same scale for optimal performance, which is often achieved by transforming the features in the range $[0, 1]$ or a standard normal distribution with zero mean and unit variance.
- Some of the selected features may be highly correlated and therefore redundant to a certain degree. In those cases, dimensionality reduction techniques are useful for compressing the features onto a lower-dimensional subspace. Reducing the dimensionality of our feature space has the advantage that less storage space is required, and the learning algorithm can run much faster.
- In certain cases, dimensionality reduction can also improve the predictive performance of a model if the dataset contains many irrelevant features (or noise); that is, if the dataset has a low signal-to-noise ratio.
- To determine whether our machine learning algorithm not only performs well on the training dataset but also generalizes well to new data, we also want to randomly divide the dataset into separate training and test datasets. We use the training dataset to train and optimize our machine learning model, while we keep the test dataset until the very end to evaluate the final model.

2. Training and selecting a predictive model

- In practice, it is therefore essential to compare at least a handful of different learning algorithms in order to train and select the best performing model.
- But before we can compare different models, we first have to decide upon a metric to measure performance.
- One commonly used metric is classification accuracy, which is defined as the proportion of correctly classified instances.
- how do we know which model performs well on the final test dataset and real-world data if we don't use this test dataset for the model selection, but keep it for the final model evaluation?
- To address the issue embedded in this question, different techniques summarized as “cross-validation” can be used. In cross-validation, we further divide a dataset into training and validation subsets in order to estimate the generalization performance of the model.

- The default parameters of the different learning algorithms provided by software libraries are optimal for our specific problem task.
- Therefore, we will make frequent use of hyperparameter optimization techniques that help us to fine-tune the performance of our model
- We can think of those hyperparameters as parameters that are not learned from the data but represent the knobs of a model that we can turn to improve its performance.

3. Evaluating models and predicting unseen data instances

- After we have selected a model that has been fitted on the training dataset, we can use the test dataset to estimate how well it performs on this unseen data to estimate the so-called generalization error.
- If we are satisfied with its performance, we can now use this model to predict new, future data.
- It is important to note that the parameters for the previously mentioned procedures, such as feature scaling and dimensionality reduction, are solely obtained from the training dataset, and the same parameters are later reapplied to transform the test dataset, as well as any new data instances
- The performance measured on the test data may be overly optimistic otherwise.

Artificial neurons – a brief glimpse into the early history of machine learning



Warren McCulloch and Walter Pitts published the first concept of a simplified brain cell, the so-called McCulloch-Pitts (MCP) neuron, in 1943.



A Logical Calculus of the Ideas Immanent in Nervous Activity by W. S. McCulloch and W. Pitts, Bulletin of Mathematical Biophysics, 5(4): 115-133, 1943.

- Frank Rosenblatt published the first concept of the perceptron learning rule based on the MCP neuron model.
- The Perceptron: A Perceiving and Recognizing Automaton by F. Rosenblatt, Cornell Aeronautical Laboratory, 1957.
- Neuron-Brain Cell
- Rosenblatt proposed an algorithm that would automatically learn the optimal weight coefficients that would then be multiplied with the input features in order to make the decision of whether a neuron fires (transmits a signal) or not.
- In the context of supervised learning and classification, such an algorithm could then be used to predict whether a new data point belongs to one class or the other.

McCulloch and Pitts described a nerve cell as a simple logic gate with binary outputs; multiple signals arrive at the dendrites, they are then integrated into the cell body, and, if the accumulated signal exceeds a certain threshold, an output signal is generated that will be passed on by the axon.

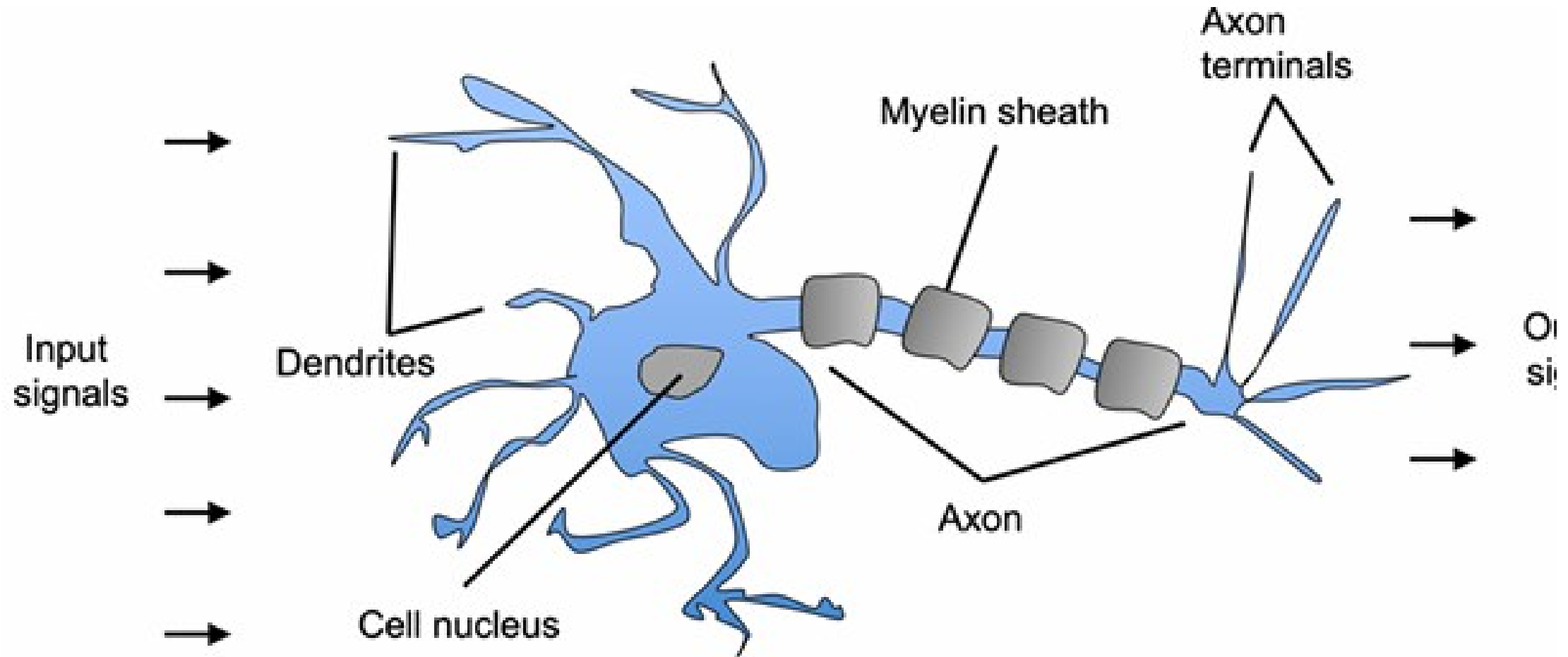


Figure 2.1: A neuron processing chemical and electrical signals

To simplify the code implementation later, we can modify this setup via a couple of steps. First, we move the threshold, θ , to the left side of the equation:

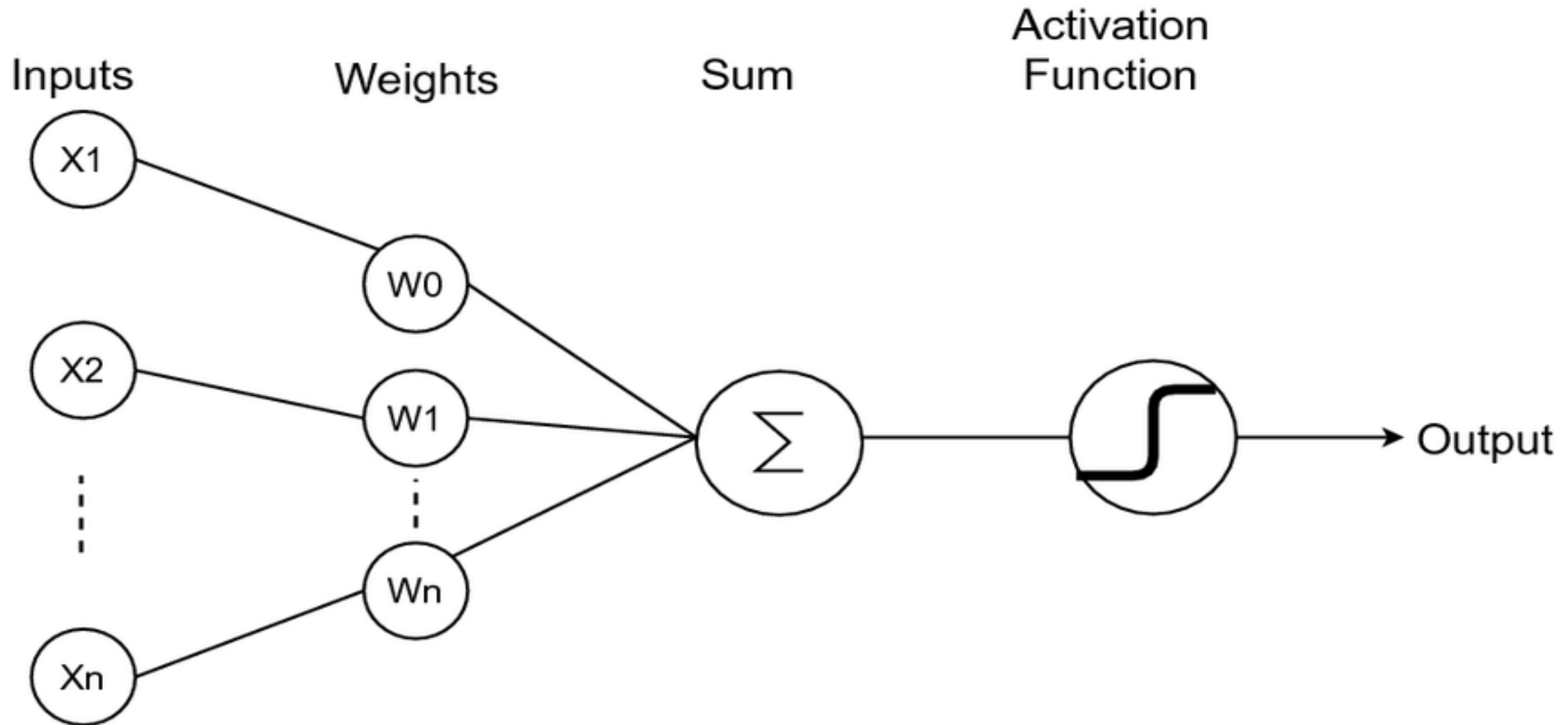
$$\begin{aligned} z &\geq \theta \\ z - \theta &\geq 0 \end{aligned}$$

Second, we define a so-called *bias unit* as $b = -\theta$ and make it part of the net input:

$$z = w_1x_1 + \dots + w_mx_m + b = \mathbf{w}^T \mathbf{x} + b$$

Third, given the introduction of the bias unit and the redefinition of the net input z above, we can redefine the decision function as follows:

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$



The formal definition of an artificial neuron

More formally, we can put the idea behind artificial neurons into the context of a binary classification task with two classes: 0 and 1. We can then define a decision function, $\sigma(z)$, that takes a linear combination of certain input values, x , and a corresponding weight vector, w , where z is the so-called net input $z = w_1x_1 + w_2x_2 + \dots + w_mx_m$:

$$\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

Now, if the net input of a particular example, $x^{(i)}$, is greater than a defined threshold, θ , we predict class 1, and class 0 otherwise. In the perceptron algorithm, the decision function, $\sigma(\cdot)$, is a variant of a unit step function:

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

Figure 2.2 illustrates how the net input $z = \mathbf{w}^T \mathbf{x} + b$ is squashed into a binary output (0 or 1) by the decision function of the perceptron (left subfigure) and how it can be used to discriminate between two classes separable by a linear decision boundary (right subfigure):

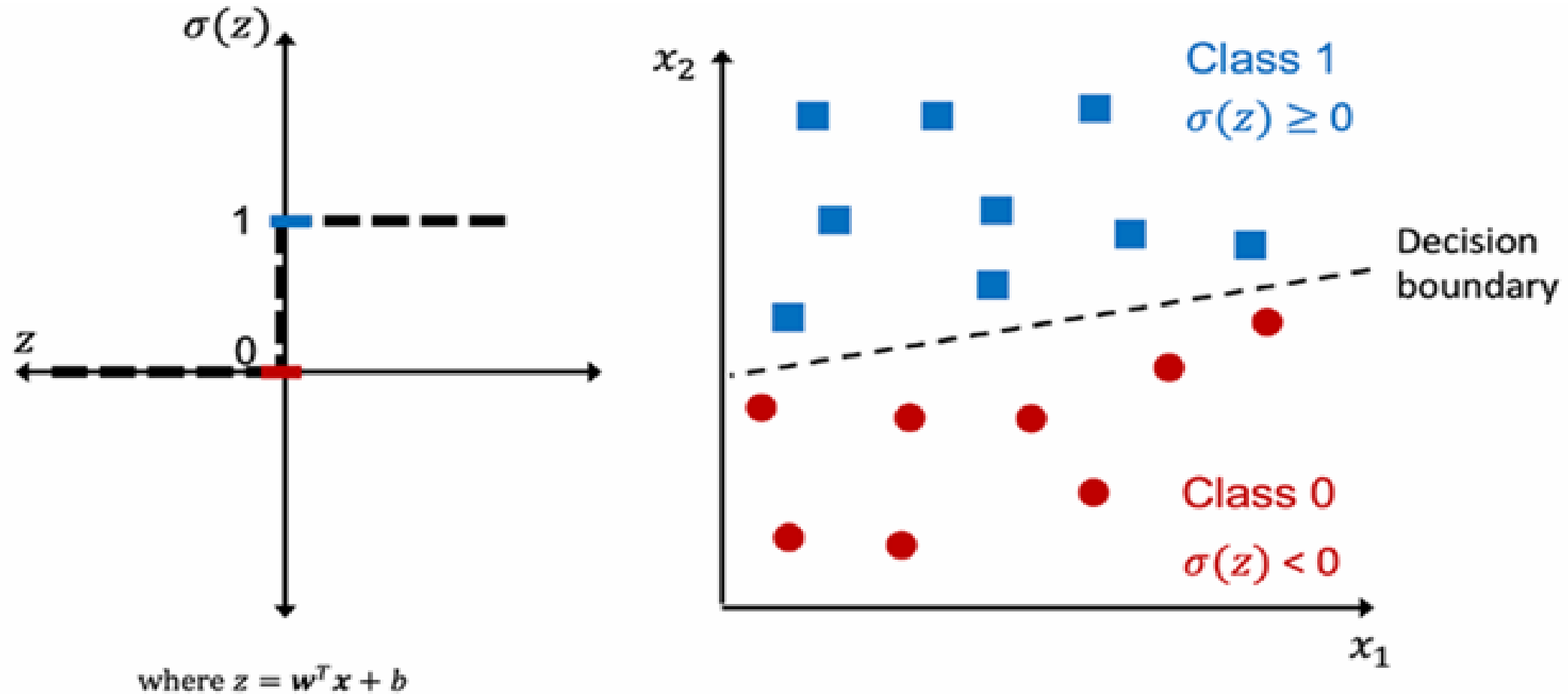


Figure 2.2: A threshold function producing a linear decision boundary for a binary classification problem

The perceptron learning rule

- The whole idea behind the MCP neuron and Rosenblatt's thresholded perceptron model is to use a reductionist approach to mimic how a single neuron in the brain works: it either fires or it doesn't. Thus, Rosenblatt's classic perceptron rule is simple,

1. Initialize the weights and bias unit to 0 or small random numbers
2. For each training example, $x^{(i)}$:
 - a. Compute the output value, $\hat{y}^{(i)}$
 - b. Update the weights and bias unit

Here, the output value is the class label predicted by the unit step function that we defined earlier, and the simultaneous update of the bias unit and each weight, w_j , in the weight vector, w , can be more formally written as:

$$w_j := w_j + \Delta w_j$$

and $b := b + \Delta b$

The update values (“deltas”) are computed as follows:

$$\Delta w_j = \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}$$

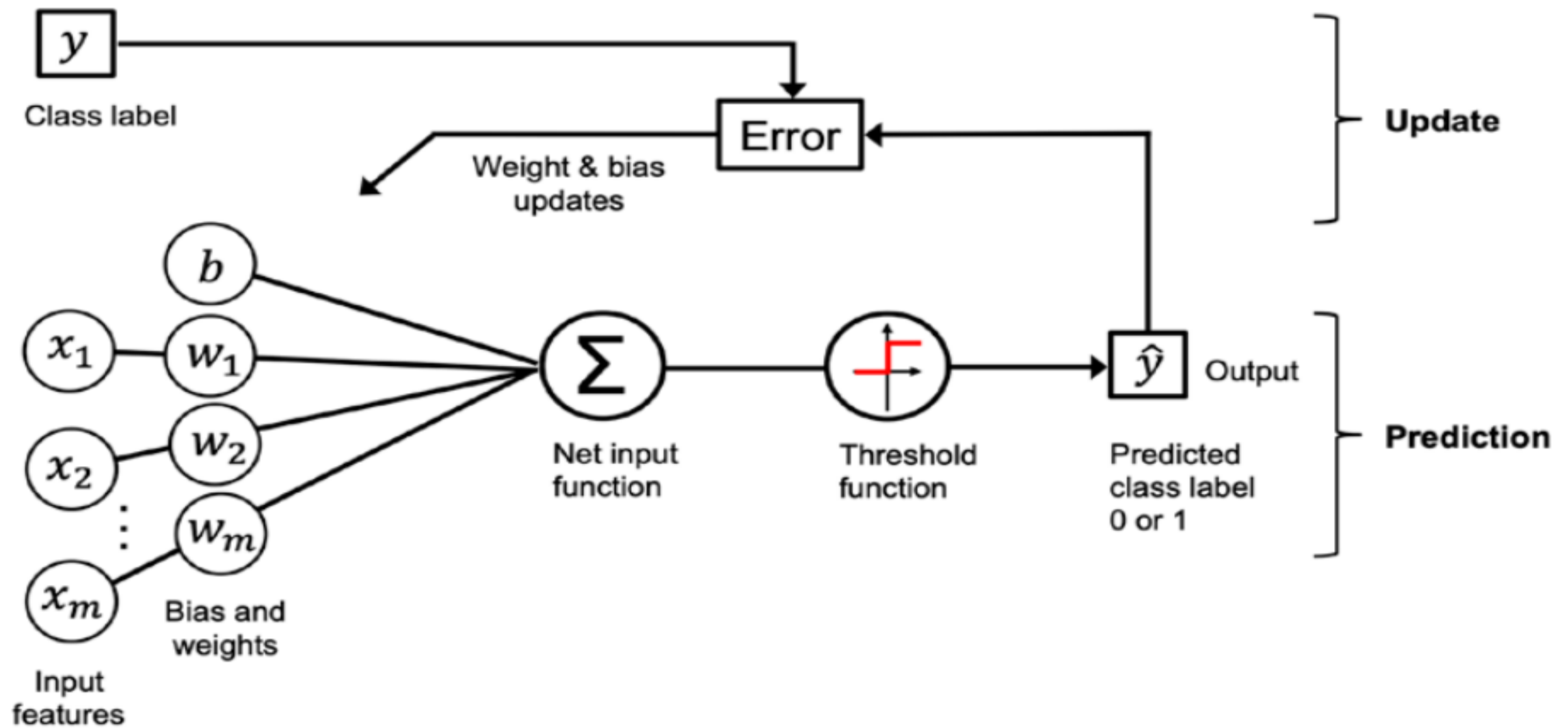
and $\Delta b = \eta(y^{(i)} - \hat{y}^{(i)})$

Note that unlike the bias unit, each weight, w_j , corresponds to a feature, x_j , in the dataset, which is involved in determining the update value, Δw_j , defined above. Furthermore, η is the learning rate (typically a constant between 0.0 and 1.0), $y^{(i)}$ is the true class label of the i th training example, and $\hat{y}^{(i)}$ is the predicted class label. It is important to note that the bias unit and all weights in the weight vector are being updated simultaneously, which means that we don't recompute the predicted label, $\hat{y}^{(i)}$, before the bias unit and all of the weights are updated via the respective update values, Δw_j and Δb . Concretely, for a two-dimensional dataset, we would write the update as:

$$\Delta w_1 = \eta(y^{(i)} - \text{output}^{(i)})x_1^{(i)};$$

$$\Delta w_2 = \eta(y^{(i)} - \text{output}^{(i)})x_2^{(i)};$$

$$\Delta b = \eta(y^{(i)} - \text{output}^{(i)})$$



The diagram illustrates how the perceptron receives the inputs of an example (x) and combines them with the bias unit (b) and weights (w) to compute the net input. The net input is then passed on to the threshold function, which generates a binary output of 0 or 1—the predicted class label of the example. During the learning phase, this output is used to calculate the error of the prediction and update the weights and bias unit.

- It is important to note that the convergence of the perceptron is only guaranteed if the two classes are linearly separable, which means that the two classes cannot be perfectly separated by a linear decision boundary.

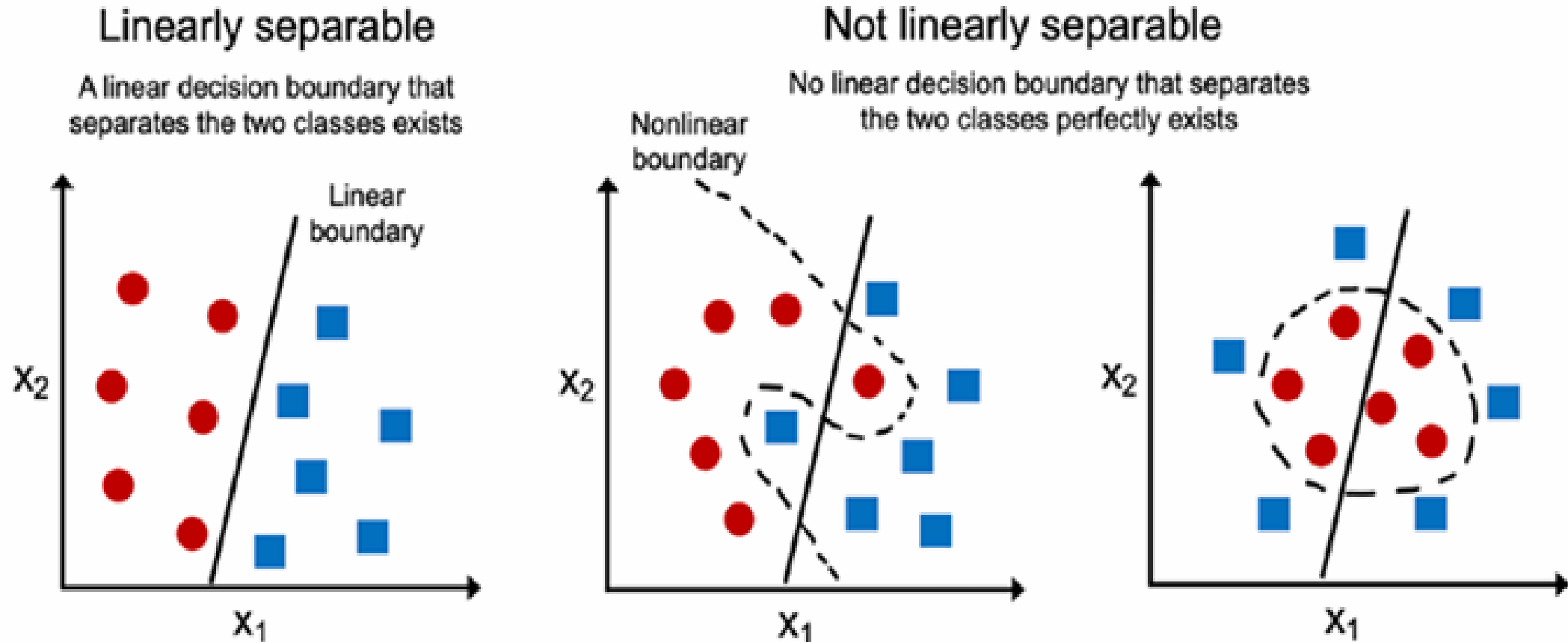


Figure 2.3: Examples of linearly and nonlinearly separable classes

Activation Functions

sigmoid function

- The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- This function outputs values between 0 and 1, making it useful for binary classification tasks.
- For negative values of z , the output is closer to 0.
- For positive values of z , the output approaches 1.
- For $z = 0$, the output is exactly 0.5.

The identity function

The identity function is mathematically represented as:

$$\sigma(z)=z$$

The output of the activation function is the same as the input, i.e., the function doesn't transform the input in any way it simply passes it through as is.

Adaline Algorithm

1. Initialization

Initialize weights, including the bias , to small random values.

2. Continuous Output:

The output is simply the weighted sum of the inputs

$$z = w_1x_1 + w_2x_2 + \dots + b$$

3. Error Calculation: Since the output is continuous, Adaline computes the error as the difference between the target value and the actual output (which is a continuous value). This error is used to update the weights using gradient descent.

The error term is:

- **Error = $y - \sigma(z)$** where y is the actual target value, and z is the net input.

4. Weight Update Rule: The weights are updated based on the error:

$$w_i \leftarrow w_i + \eta \cdot \text{Error} \cdot x_i$$

η -learning rate

x_i is the corresponding input.

5. Repeat steps 2,3 and 4 till algorithm converges(until the cost function converges).

- During the training phase, Adaline does not discretize its output immediately. Instead of making a binary decision (0 or 1), it produces a continuous output.
- This continuous nature of the output and weight adjustment based on the **magnitude of the error** is what allows Adaline to converge more smoothly in comparison to the Perceptron.

```
class Adaline:
    def __init__(self, learning_rate=0.01, epochs=10):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None
    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        for _ in range(self.epochs):
            for idx, x_i in enumerate(X):
                linear_output = np.dot(x_i, self.weights) + self.bias
                y_pred = linear_output # Linear activation (identity function)
                error = y[idx] - y_pred
                self.weights += self.learning_rate * error * x_i
                self.bias += self.learning_rate * error
    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return np.where(linear_output >= 0, 1, 0) # Apply step function to continuous output
```

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])  
y = np.array([0, 0, 0, 1])
```

```
adaline = Adaline(learning_rate=1.519, epochs=10)  
adaline.fit(X, y)  
predictions = adaline.predict(X)
```

```
print("Adaline Predictions:", predictions)  
print("Weights:", adaline.weights)  
print("Bias:", adaline.bias)
```

- **Gradient Descent is an optimization algorithm** used to minimize the cost function of a model, primarily in machine learning and deep learning.
- **Gradient Descent iteratively adjusts model parameters to find the optimal values that minimize the error between the model's predictions and the actual target values.**
- **Weights and biases** in each layer are the parameters.
- **Parameters** determine how inputs are transformed as they pass through layers of neurons to make predictions.
- The **learning process** adjusts these weights and biases to reduce the overall error.
- **model parameters** are the variables that the algorithm learns and adjusts during training to minimize the cost function, ultimately improving the model's predictions.

The parameter changes, $\Delta \mathbf{w}$ and Δb , are defined as the negative gradient multiplied by the learning rate, η :

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L(\mathbf{w}, b), \quad \Delta b = -\eta \nabla_b L(\mathbf{w}, b)$$

To compute the gradient of the loss function, we need to compute the partial derivative of the loss function with respect to each weight, w_j :

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)}$$

Similarly, we compute the partial derivative of the loss with respect to the bias as:

$$\frac{\partial L}{\partial b} = -\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right)$$

$$\Delta w_j = -\eta \frac{\partial L}{\partial w_j} \text{ and } \Delta b = -\eta \frac{\partial L}{\partial b}$$

$$\begin{aligned} \frac{\partial L}{\partial w_j} &= \frac{\partial}{\partial w_j} \frac{1}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right)^2 = \frac{1}{n} \frac{\partial}{\partial w_j} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right)^2 \\ &= \frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) \frac{\partial}{\partial w_j} \left(y^{(i)} - \sigma(z^{(i)}) \right) \\ &= \frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) \frac{\partial}{\partial w_j} \left(y^{(i)} - \sum_i \left(w_j x_j^{(i)} + b \right) \right) \\ &= \frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) \left(-x_j^{(i)} \right) = -\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)} \end{aligned}$$

The same approach can be used to find partial derivative $\frac{\partial L}{\partial b}$ except that $\frac{\partial}{\partial b} \left(y^{(i)} - \sum_i \left(w_j^{(i)} x_j^{(i)} + b \right) \right)$ is equal to -1 and thus the last step simplifies to $-\frac{2}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right)$.

- The goal of gradient descent is to minimize this cost function.

Gradient:

- **The gradient is the vector of partial derivatives of the cost function with respect to each model parameters (weights and bias).**
- It points in the direction of the steepest increase of the cost function.
- To minimize the cost function, we want to move in the opposite direction, i.e., the direction of the negative gradient.
- The core idea of gradient descent is to update the weights in the direction that reduces the cost function

Gradient Descent Algorithm

1. **Initialize Parameters:** Start with random values for model parameters (weights and biases).
2. **Calculate the Cost:** Use the model to predict outcomes, then calculate the cost (error) between predictions and actual values.
3. **Compute the Gradient:** Find the gradient (slope) of the cost function with respect to each parameter. The gradient shows the direction of the steepest ascent.
4. **Update Parameters:** Update the parameters by moving in the opposite direction of the gradient. The step size is controlled by the learning rate.
5. **Repeat:** Repeat steps 2–4 until the cost function reaches a minimum or stops improving significantly.

1. Feature Weight Update:

$$w_j := w_j + \alpha \sum_{i=1}^n (y_i - \hat{y}_i) x_{ij}$$

2. Bias Update:

$$w_0 := w_0 + \alpha \sum_{i=1}^n (y_i - \hat{y}_i)$$

3. Cost Calculation (Mean Squared Error):

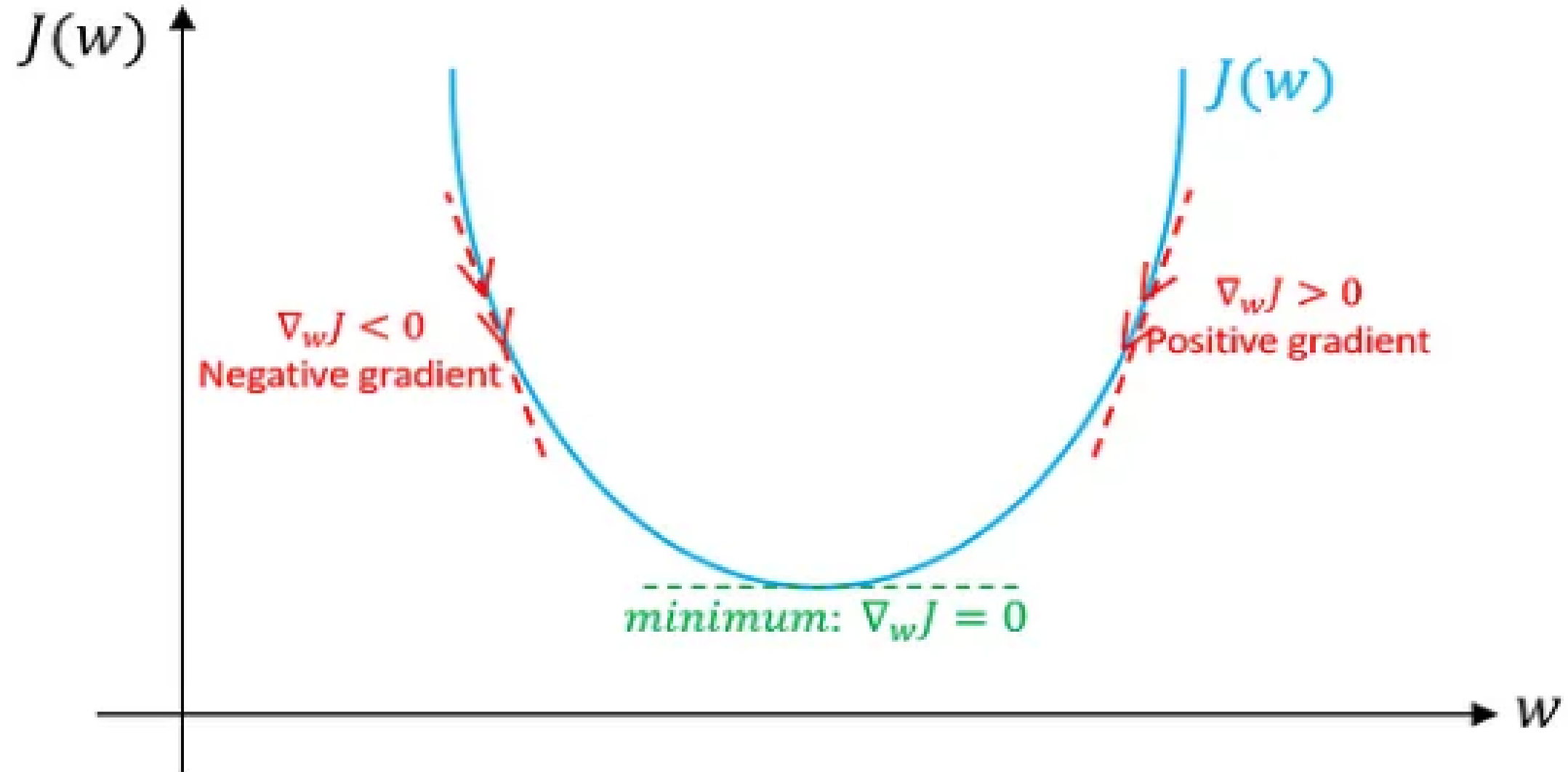
$$J(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The main idea behind gradient descent is climbing down a hill until a global loss minimum is reached. In each iteration, we take a step in the opposite direction of the gradient, where the step size is determined by the value of the learning rate, as well as the slope of the gradient

Using gradient descent, we can now update the model parameters by taking a step in the opposite direction of the gradient, ∇L of our loss function, $L(\mathbf{w}, b)$

$$L(\mathbf{w}, b) = \frac{1}{2n} \sum_{i=1}^n \left(y^{(i)} - \sigma(z^{(i)}) \right)^2$$

$$J(w) = \frac{1}{2}(y - z)^2$$



- **Update Rule:**

- The core idea of gradient descent is to update the weights in the direction that reduces the cost function.
- For each weight w_i the weight update rule is:

$$w_i \leftarrow w_i - \eta \frac{\partial J(w)}{\partial w_i}$$

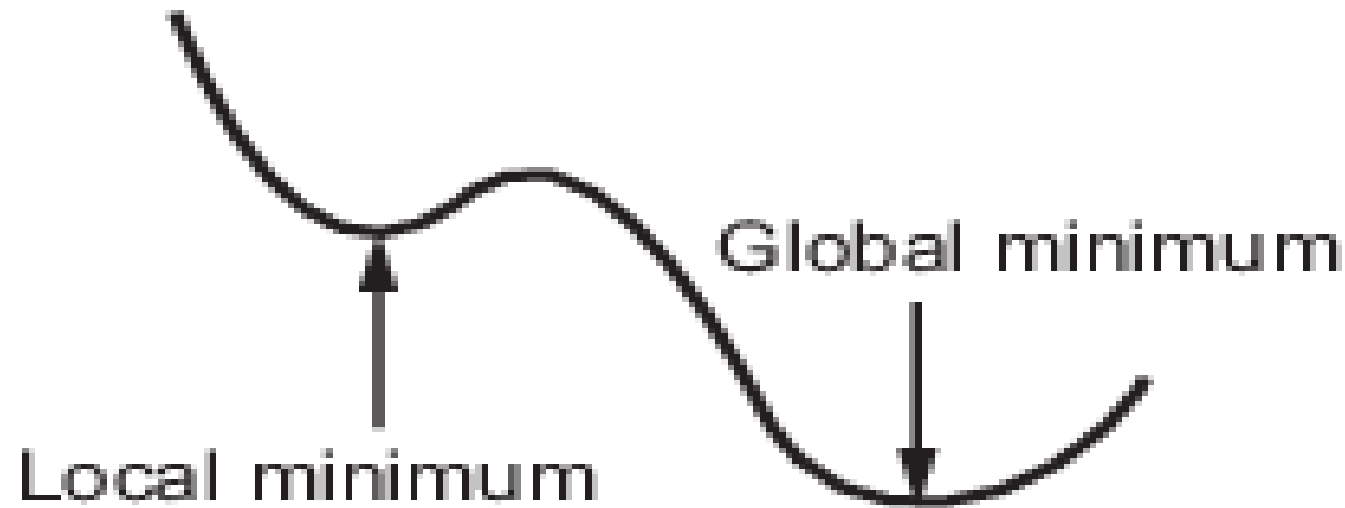
where η is the **learning rate**, a small positive value that controls how large a step is taken in the direction of the negative gradient.

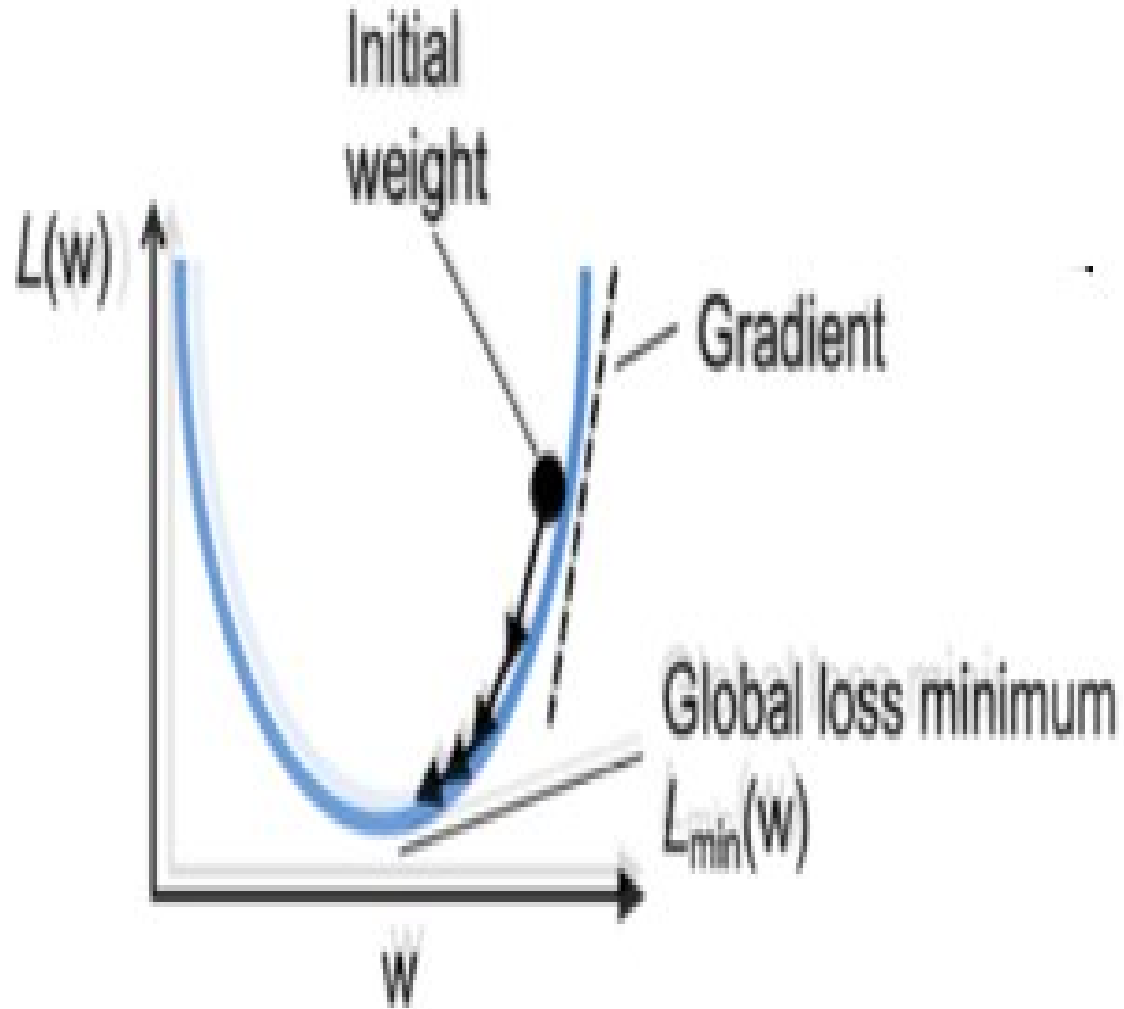
- The learning rate controls the speed of convergence. If it's too small, the algorithm may converge too slowly, and if it's too large, it might overshoot the minimum.

Learning Process-Gradient descent works iteratively

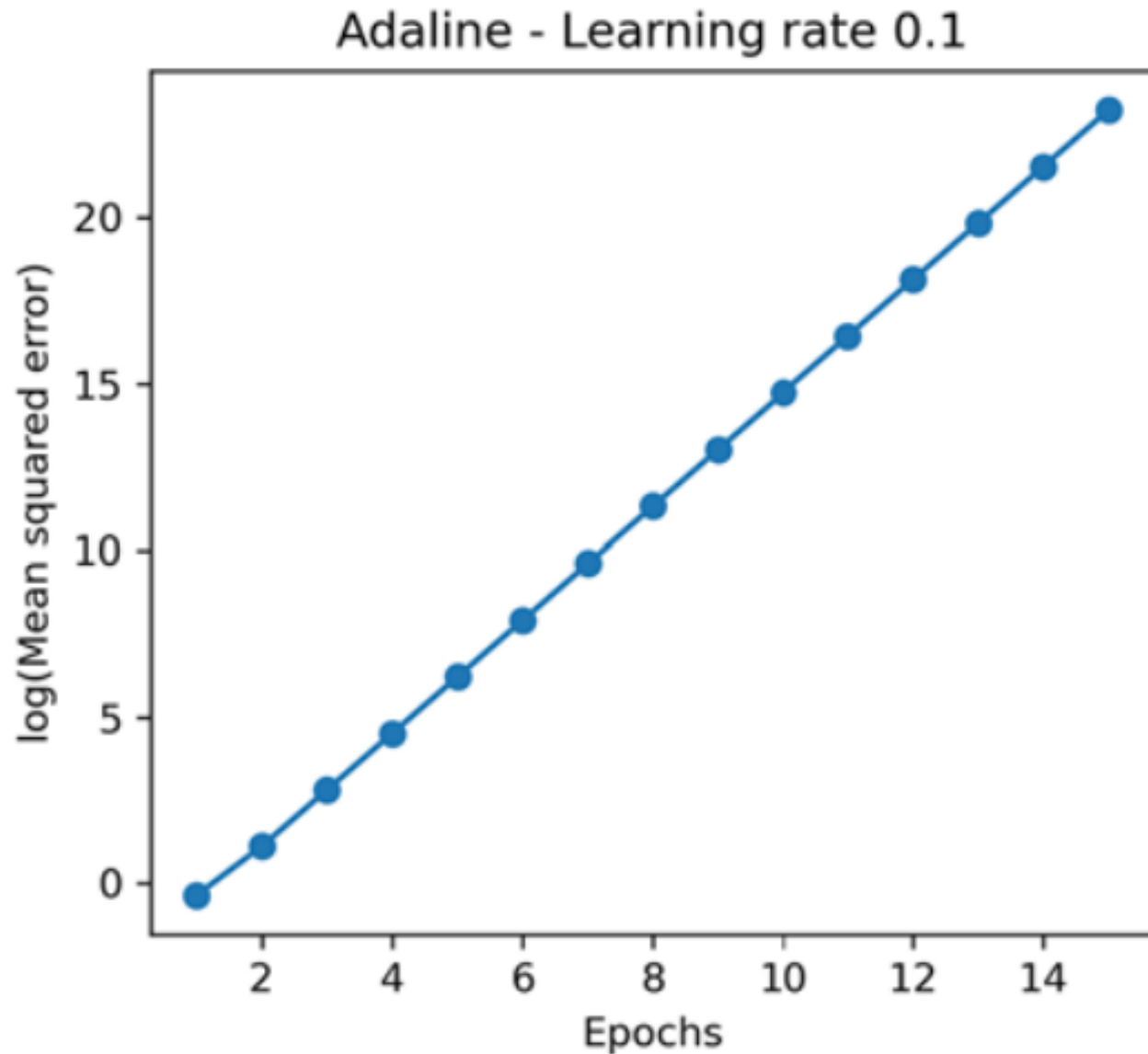
- **Calculate the gradient of the cost function with respect to the weights.**
- **Update the weights by moving in the opposite direction of the gradient.**
- **Repeat this process until the cost function converges to a minimum.**
(i.e., no significant changes in the cost function across iterations).

- A local minimum of a function is a point where the function value is smaller than at nearby points, but possibly greater than at a distant point.
- A global minimum is a point where the function value is smaller than at all other feasible points.





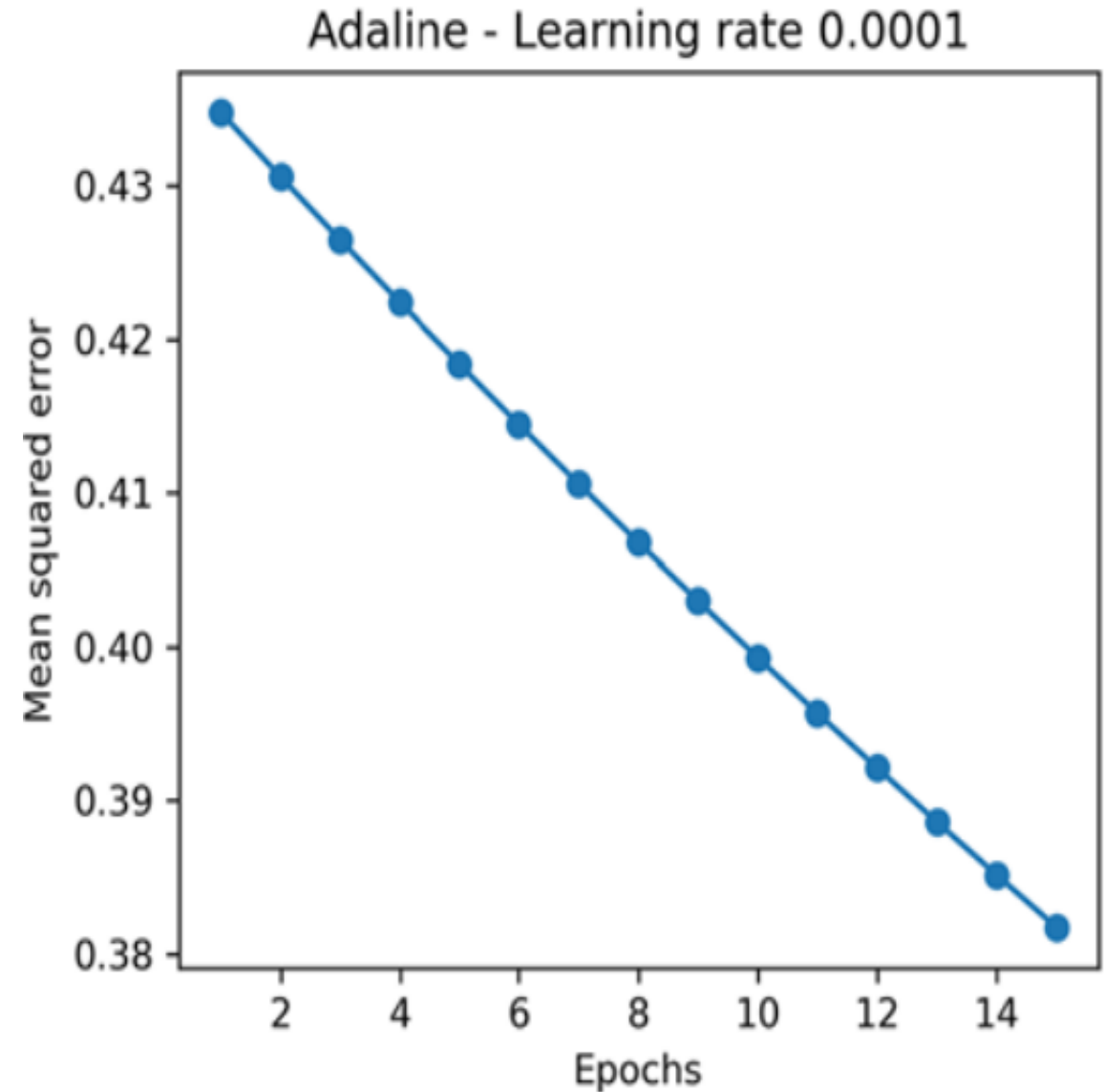
- if we change the value of a particular weight parameter to minimize the loss function, L .
- The left subfigure illustrates the case of a well-chosen learning rate,
- where the loss decreases gradually, moving in the direction of the global minimum.



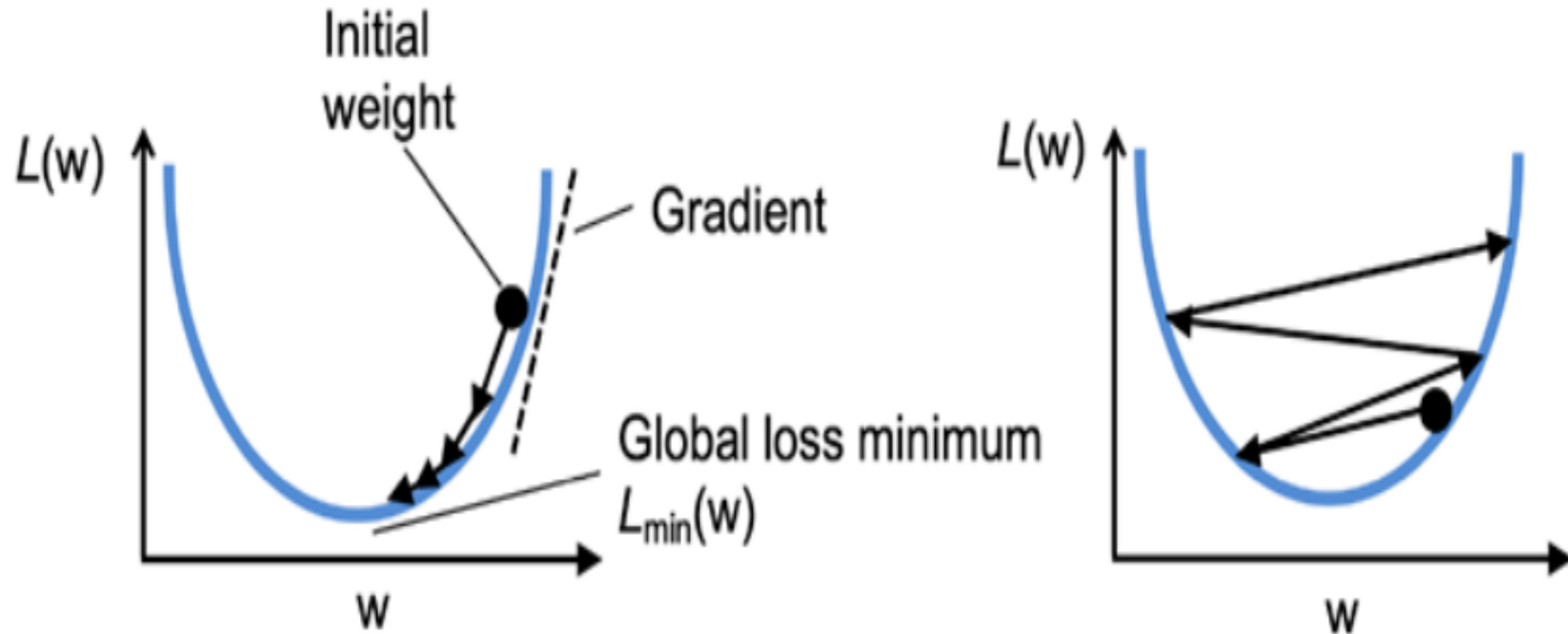
The left chart shows what could happen if we choose a learning rate that is too large.

Instead of minimizing the loss function, the MSE becomes larger in every epoch, because we overshoot the global minimum.

- we can see that the loss decreases on the right plot, but the chosen learning rate, η 0.0001, is so small that the algorithm would require a very large number of epochs to converge to the global loss minimum



A comparison of a well-chosen learning rate and a learning rate that is too large



Stochastic Gradient Descent (SGD)

- In Stochastic Gradient Descent, the weights are updated after every single training example.
- It's faster and more efficient for large datasets but can result in noisy updates due to the small batch size (just one example).
- SGD typically fluctuates around the minimum rather than converging smoothly.

- Stochastic Gradient Descent (SGD) is a variation of the gradient descent algorithm used in machine learning and optimization tasks, particularly for training neural networks.
- While standard gradient descent computes the gradient of the cost function using the entire training dataset, SGD updates the model's parameters based on a single randomly selected training example at a time.

$$\Delta w_j = \frac{2\eta}{n} \sum_i \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)}$$

we update the parameters incrementally for each training example, for instance:

$$\Delta w_j = \eta \left(y^{(i)} - \sigma(z^{(i)}) \right) x_j^{(i)}, \quad \Delta b = \eta \left(y^{(i)} - \sigma(z^{(i)}) \right)$$

- Although SGD can be considered as an approximation of gradient descent, it typically reaches convergence much faster because of the more frequent weight updates.
- Each gradient is calculated based on a single training example, the error surface is noisier than in gradient descent, which can also have the advantage that SGD can escape shallow local minima

Advantage

- SGD can be used for online learning.
- In online learning, our model is trained on the fly as new training data arrives.
- This is especially useful if we are accumulating large amounts of data, for example, customer data in web applications.
- Using online learning, the system can immediately adapt to changes, and the training data can be discarded after updating the model if storage space is an issue.

Feature Scaling

- It is an essential preprocessing step for gradient descent, making it more effective by improving convergence and optimizing the learning rate.
- Gradient descent relies on moving towards the minimum of the cost function by adjusting parameters based on gradients.
- When features have vastly different ranges, the cost function's contour can become elongated and skewed, making the gradient descent path inefficient.

Why Feature Scaling?

- **Slow Convergence:** Larger ranges in feature values can cause the gradient descent to take more time to converge, as the algorithm makes smaller steps in some dimensions and larger ones in others.
- **Risk of Getting Stuck:** With unscaled features, gradient descent may “oscillate” across the cost surface, increasing the risk of missing the optimal solution.
- **Difficulties with Optimal Learning Rates:** Without scaling, finding a learning rate that works for all features becomes challenging. A learning rate that works well for one feature might not be suitable for another with a larger range.

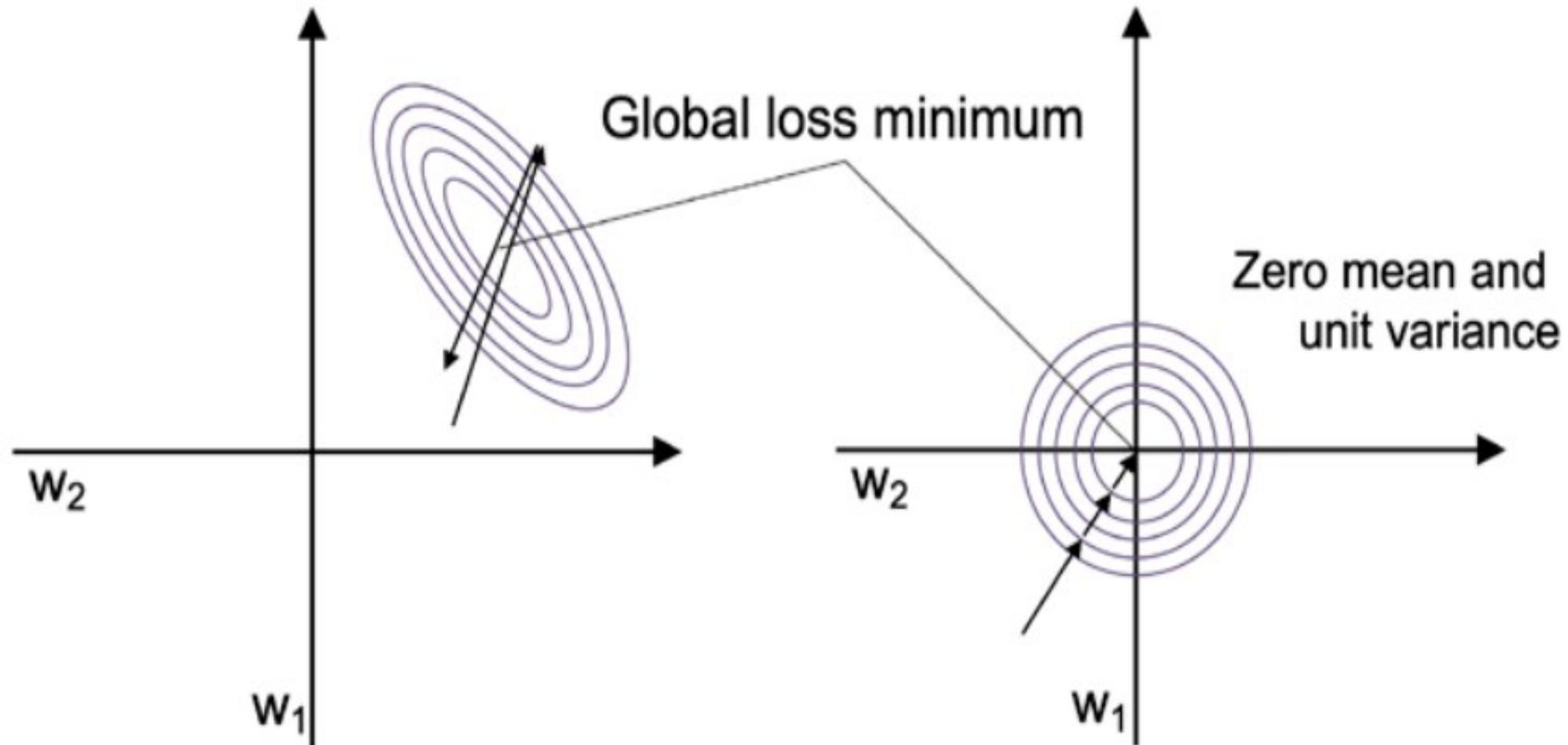
Standardization feature scaling technique

- This normalization procedure helps gradient descent learning to converge more quickly, however, it does not make the original dataset normally distributed.
- Standardization shifts the mean of each feature so that it is centered at zero and each feature has a standard deviation of 1 (unit variance).
- For instance, to standardize the j th feature, we can simply subtract the sample mean, μ_j , from every training example and divide it by its standard deviation, σ .

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

- x^j is a vector consisting of the j^{th} feature values of all training examples, n , and this standardization technique is applied to each feature, j , in our dataset.

A comparison of unscaled and standardized features on gradient updates



Difference between Gradient Descent and Stochastic Gradient Descent

Gradient Descent and Stochastic Gradient Descent (SGD) are optimization algorithms used to minimize the cost function in machine learning and deep learning models. They differ primarily in how they compute and use the gradient.

Feature	GD	SGD
Gradient Computation	Entire dataset	Single sample at a time
Convergence Speed	slower	Faster for large datasets
Stability	Smooth and stable updates	Noisy updates with fluctuations
Computational Cost	High (requires all data)	Low (one sample at a time)
Memory Requirements	High (entire dataset in memory)	Low (single sample in memory)
Escaping Local Minima	May get stuck in local minima	Better at escaping local minima

Gradient Descent

- It computes the gradient of the cost function with respect to the parameters using the entire training dataset.
- Compute the gradient for the entire dataset.
- Update the model parameters using the gradient.
- Update Rule

$$\theta = \theta - \eta \nabla J(\theta)$$

- θ : Parameters of the model.
- η : Learning rate.
- $\nabla J(\theta)$: Gradient of the cost function $J(\theta)$.

Characteristics

- **Deterministic:** Produces the same updates for the same dataset.
- **Slow Updates:** Since it processes the entire dataset, it is computationally expensive, especially for large datasets.
- **Global Convergence:** Provides a smooth path to the minimum but can get stuck in local minima.

Stochastic Gradient Descent (SGD)

SGD computes the gradient of the cost function using a single random sample at a time.

Steps:

- Select one random data point from the training set.
- Compute the gradient for that single sample.
- Update the model parameters.

$$\theta = \theta - \eta \nabla J(\theta; x^{(i)}, y^{(i)})$$

Where $(x^{(i)}, y^{(i)})$: A single data point and its label.

Characteristics

- **Faster Updates:** Updates are made after processing each sample, making it faster for large datasets.
- **Noisy Updates:** Updates fluctuate due to randomness, which can help escape local minima.
- **Non-Deterministic:** Each run may produce slightly different results because of random sampling.
- **Less Memory Intensive:** Only a single sample is required in memory at any time